



Trabajo Práctico N^o2: Filtrado digital de señales

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Julio de 2025

Resumen

Realizar un programa aplicativo que sea capaz de aplicar un filtro FIR, por muestras, a un buffer de memoria de 512 muestras, que es adquirido con el Laboratorio 1 teniendo en cuenta las frecuencias de muestreo de 8 kHz, 16 kHz, 24 kHz, 40 kHz y 48 kHz. Con una de las teclas de la placa de evaluación STM32F446RE, se habilitara la aplicación del filtro o se hace bypass del filtro a un buffer de salida distinto del buffer de entrada. De este buffer de salida se enviaran las muestras al DAC de la palca STM32F446RE. Visualizar el resultado utilizando un osciloscopio y comparar los resultados.

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
1.2. Aspectos a tener en cuenta	3
2. Marco teórico	4
2.1. Conceptos fundamentales del filtrado digital	4
2.2. Filtros FIR	4
2.2.1. Diseño de filtros FIR y el uso de ventanas	5
2.3. Filtros IIR	6
2.3.1. Filtros biquad y cascada de secciones	6
2.4. Implementación con la biblioteca CMSIS-DSP	7
3. Placa de Desarrollo y Herramientas Utilizadas	8
3.1. Microcontrolador STM32F446RE	8
3.2. Periféricos clave utilizados	9
3.2.1. ADC (Analog-to-Digital Converter)	9
3.2.2. DAC (Digital-to-Analog Converter)	9
3.2.3. DMA (Direct Memory Access)	9
3.2.4. TIM (Timers)	10
3.2.5. GPIO – General Purpose Input/Output	10
3.2.6. Herramientas de desarrollo	10
4. Calculo y diseño de filtros	11
5. Diseño del programa	14
6. Programa principal	15
6.1. Implementación del main.c	15
6.2. Configuración de periféricos	22
6.2.1. Configuración del ADC	22
6.2.2. Configuración del DAC	22
6.2.3. Configuración del temporizador	23
6.2.4. Configuración del DMA	23
6.2.5. Configuración de GPIO	23
6.2.6. Inicialización de los filtros digitales	24
6.2.7. Ejecución en tiempo real	24
7. Resultados experimentales	24
7.1. Filtro pasa bajo	24
7.2. Filtro pasa alto	27
7.3. Filtro pasa banda	29
7.4. Filtro rechaza banda	32
8. Conclusiones	34

1. Introducción

El procesamiento digital de señales (DSP) en tiempo real es una disciplina fundamental en aplicaciones embebidas modernas, tales como audio, telecomunicaciones, sensorado inteligente, etc. Implementar estos sistemas directamente sobre microcontroladores con recursos limitados impone desafíos tanto computacionales como de diseño. En este contexto, el presente proyecto busca implementar un sistema completo de adquisición, procesamiento y reproducción de señales analógicas en tiempo real, utilizando el microcontrolador STM32F446RE, el cual cuenta con arquitectura ARM Cortex-M4 y soporte para operaciones DSP optimizadas.

Este trabajo continúa el desarrollo iniciado en el primer proyecto. En esta segunda parte, se incorporan capacidades de filtrado digital, trabajando en formato Q15 (punto fijo) para garantizar eficiencia y compatibilidad con la arquitectura.

1.1. Objetivos del proyecto

El sistema conserva la misma arquitectura base del laboratorio anterior —adquisición ADC, procesamiento intermedio, y reproducción DAC—, pero ahora añade la posibilidad de aplicar filtros digitales en tiempo real. Los tipos de filtros implementados incluyen paso bajo (LP), paso alto (HP), pasa banda (BP) y rechazo de banda (BS). El usuario puede seleccionar dinámicamente tanto el tipo de filtro como la frecuencia de muestreo mediante interrupciones externas por GPIO (pulsadores).

<i>Tipo</i>	f_{c1}	f_{c2}	f_0	BW
Paso bajo	3600 Hz	—	—	—
Paso alto	—	35 Hz	—	—
Pasa banda	35 Hz	3500 Hz	—	—
Rechazo de banda	—	—	50 Hz	15 Hz

Cuadro 1: Parámetros de diseño para cada tipo de filtro

Respecto a los requerimientos de atenuación en la banda de rechazo de los filtros, debe estar en el orden de los 25 a 30 dB.

1.2. Aspectos a tener en cuenta

- Para lograr un sistema de procesamiento en tiempo real, se mantienen las mismas técnicas que el laboratorio anterior enfocadas en reducir la latencia del sistema. En primer lugar, se utiliza un esquema de doble buffer mediante DMA para la transferencia eficiente de datos entre el ADC/DAC y la memoria RAM, evitando cuellos de botella en la ejecución. La arquitectura ARM Cortex-M4 de la STM32F446RE permite aprovechar la unidad de punto fijo con saturación (DSP extension) y el contador de ciclos DWT, lo cual resulta clave para medir el rendimiento del sistema.
- Otro aspecto crucial es el correcto escalado y normalización de los datos. La señal adquirida del ADC de 12 bits es convertida a formato Q15 para ser procesada por la biblioteca CMSIS-DSP, y luego reconvertida al rango del DAC, que también es de 12 bits.

- Se debe contemplar que el sistema debe operar a distintas frecuencias de muestreo (8 kHz, 16 kHz, 24 kHz, 40 kHz y 48 kHz), lo cual exige una correcta reconfiguración del temporizador de disparo y la selección de coeficientes adecuados para cada filtro y frecuencia (el cambio de frecuencia de muestreo cambia los coeficientes, por mas que el filtro sea el mismo tipo). El sistema cuenta también con retroalimentación visual mediante LED RGB para indicar el estado actual del muestreo, y contempla un modo de parada segura (modo 5) para evitar errores o conflictos cuando se reconfigura dinámicamente el sistema.
- Por ultimo el calculo y diseño de los filtros debe apegarse a los requerimientos mencionados en el cuadro anterior. Como se explica posteriormente en la sección 4, no es posible su implementación con los filtros de tipo FIR, por lo que se opto por implementarlos con filtros IIR, estos tipos de filtros son capaces de cumplir con las especificaciones de frecuencia de corte y atenuación en las bandas, a expensas de perder linealidad en la fase y poder tener problemas de estabilidad.

2. Marco teórico

El filtrado digital es una operación fundamental dentro del procesamiento digital de señales (DSP), utilizada para modificar el contenido espectral de una señal, suprimiendo componentes no deseados o destacando frecuencias de interés. En sistemas embebidos de tiempo real, como los basados en microcontroladores ARM Cortex-M, es indispensable implementar filtros eficientes que cumplan con los requerimientos funcionales sin comprometer el rendimiento del sistema.

2.1. Conceptos fundamentales del filtrado digital

Un filtro digital es un sistema que transforma una señal de entrada $x[n]$ en una señal de salida $y[n]$, aplicando una operación matemática determinada. Los filtros pueden clasificarse en diversas categorías según su comportamiento en frecuencia (paso bajo, paso alto, pasa banda, rechazo de banda) y según su estructura (FIR o IIR). Los objetivos típicos del filtrado son:

- Atenuar ruido o interferencias.
- Extraer información útil dentro de una banda específica.
- Suavizar o eliminar fluctuaciones rápidas en la señal.
- Adaptar el contenido espectral de la señal para análisis posterior.

2.2. Filtros FIR

Filtros FIR (Finite Impulse Response): Son filtros cuya respuesta al impulso es finita, ya que dependen únicamente de muestras pasadas de la señal de entrada. Su estructura general es:

$$y[n] = \sum_{k=0}^N b_k x[n-k] \quad (1)$$

Tienen las siguientes características:

- Siempre estables.
- Fase lineal.
- Requieren más coeficientes para lograr altas pendientes de atenuación.

2.2.1. Diseño de filtros FIR y el uso de ventanas

El diseño de filtros FIR (Finite Impulse Response) parte, generalmente, de una respuesta ideal en frecuencia. Por ejemplo, un filtro paso bajo ideal tendría una transición perfectamente abrupta entre la banda pasante y la banda atenuada, lo cual implica una respuesta al impulso infinita. Como esto no es realizable en sistemas discretos, se realiza una truncación temporal de esa respuesta, lo cual equivale a multiplicarla por una ventana rectangular en el dominio temporal. Sin embargo, esta truncación introduce efectos no deseados en la respuesta en frecuencia, principalmente el llamado fenómeno de Gibbs.

Fenómeno de Gibbs: Es una oscilación persistente (rizado o *ripple*) que aparece cerca de las discontinuidades en la respuesta en frecuencia del filtro debido a la truncación abrupta de la respuesta al impulso, este efecto provoca:

- Rizado en la banda pasante y/o banda de detención.
- Ensanchamiento de la zona de transición.
- Picos de sobrepaso que no desaparecen al aumentar el número de coeficientes (aunque se concentran en un área más estrecha).

Para mitigar el fenómeno de Gibbs y mejorar el comportamiento espectral del filtro, se utilizan funciones ventana, que consisten en aplicar un suavizado gradual a los extremos de la respuesta al impulso truncada. Este procedimiento reduce el rizado a costa de un ensanchamiento adicional en la zona de transición, representando un compromiso entre precisión en frecuencia y suavidad espectral, las ventanas mas comunes son:

- **Hamming:** Buena atenuación lateral con transiciones moderadas.
- **Hann:** Similar a Hamming pero con menor atenuación lateral.
- **Blackman:** Alta atenuación lateral, pero con una transición más ancha.
- **Rectangular:** Equivale a no aplicar ventana. Genera el mayor rizado (fenómeno de Gibbs más pronunciado).

La elección de la ventana depende de las prioridades del diseño: una mejor atenuación fuera de banda o una transición más angosta. Es importante destacar que este método de diseño con ventanas es exclusivo de filtros FIR, ya que los filtros IIR se diseñan a partir de aproximaciones analíticas (como Butterworth o Chebyshev) y no requieren truncación ni uso de ventanas.

Ventana	Ancho de banda	Atenuación lateral	Uso típico
Rectangular	Estrecho (mejor)	≈ -13 dB	Alta resolución, pero máximo ripple (Gibbs marcado)
Hamming	Moderado	≈ -43 dB	Buen compromiso entre transición y atenuación
Hann	Moderado	≈ -31 dB	Menor ripple que Hamming, pero peor resolución
Blackman	Ancho (peor)	≈ -58 dB	Muy buena supresión lateral, transición más ancha
Kaiser (β variable)	Ajustable	Ajustable	Permite ajustar entre resolución y atenuación según β

Cuadro 2: Características principales de funciones ventana comunes en diseño FIR

2.3. Filtros IIR

Filtros IIR (Infinite Impulse Response): se caracterizan por poseer una respuesta al impulso de duración infinita. Esto se debe a que incluyen términos de retroalimentación, es decir, la salida actual del sistema depende tanto de entradas presentes y pasadas como de salidas pasadas. Su expresión general en forma de ecuación en diferencias se escribe como:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k] \quad (2)$$

Características principales:

- Requieren menos coeficientes que un FIR para lograr especificaciones similares.
- Son más eficientes computacionalmente.
- Pueden presentar problemas de estabilidad si no se diseñan correctamente.

2.3.1. Filtros biquad y cascada de secciones

En la implementación práctica de filtros IIR, especialmente en sistemas digitales con aritmética de punto fijo como los microcontroladores, es habitual descomponer el filtro en secciones de segundo orden denominadas biquads. Cada biquad corresponde a un filtro de segundo orden cuya estructura es más robusta frente a errores numéricos. La ecuación que describe una sección biquad es:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2] \quad (3)$$

Para filtros de orden superior, varias de estas secciones se encadenan en forma de cascada. Esta estrategia permite controlar mejor la ganancia interna del sistema, aplicar escalado para prevenir desbordamientos (post-shift), y facilitar el ajuste de parámetros individuales. Además, mejora la estabilidad numérica general del filtro, lo que es especialmente importante en contextos de baja resolución, como en procesadores digitales de señales con punto fijo.

A diferencia de los filtros FIR, que son siempre estables si sus coeficientes son finitos, los filtros IIR pueden volverse inestables debido a la presencia de realimentación. La estabilidad de un filtro IIR depende de la ubicación de los polos de su función de transferencia en el plano- z ; para que sea estable, todos los polos deben estar dentro del círculo unitario. Esto implica que el diseño de filtros IIR requiere un mayor cuidado en la selección de coeficientes, especialmente en implementaciones con precisión limitada, como las de punto fijo. En contraste, los filtros FIR no presentan este tipo de limitaciones estructurales, lo que los hace más robustos desde el punto de vista de la estabilidad, aunque a costa de una mayor complejidad computacional para alcanzar el mismo rendimiento en frecuencia.

2.4. Implementación con la biblioteca CMSIS-DSP

La biblioteca CMSIS-DSP, desarrollada por ARM, proporciona un conjunto de funciones optimizadas para realizar procesamiento digital de señales en microcontroladores con núcleo Cortex-M. Su diseño está pensado para maximizar el rendimiento utilizando instrucciones SIMD cuando están disponibles, y ofrece soporte tanto para punto fijo (Q15) como para punto flotante (float32). Entre sus herramientas más destacadas se encuentran funciones específicas para el diseño y ejecución de filtros FIR e IIR.

Para implementar un filtro IIR en cascada de secciones biquad utilizando CMSIS-DSP en formato Q15, es fundamental seguir una secuencia precisa de pasos. En primer lugar, se deben calcular los coeficientes del filtro, los cuales deben estar organizados en bloques de cinco valores por sección biquad: tres coeficientes para la parte directa (**b0**, **b1**, **b2**) y dos para la parte recursiva (**a1**, **a2**). Estos coeficientes deben ser previamente escalados y convertidos al formato Q15, lo cual puede hacerse con herramientas como MATLAB, Python (SciPy) o incluso manualmente si el diseño es simple. El siguiente paso consiste en preparar la estructura de datos `arm_biquad_casd_df1_inst_q15`, la cual describe el filtro. Esta estructura debe contener el número de etapas, un puntero al arreglo de coeficientes, un puntero al buffer de estado (de longitud $4 \times \text{numStages}$) y el parámetro `postShift`, que ajusta el escalado de salida para prevenir saturación en aritmética de punto fijo. Con estos datos definidos, se debe llamar a la función `arm_biquad_cascade_df1_init_q15`, que inicializa la estructura asociando los arreglos y parámetros con los campos internos del filtro. Este paso es obligatorio antes de aplicar el procesamiento.

Una vez inicializado, el filtro puede aplicarse sobre un bloque de muestras utilizando la función `arm_biquad_cascade_df1_q15`, la cual toma como argumentos la estructura, un puntero al arreglo de entrada, otro al de salida, y la cantidad de muestras. Esta función recorre cada muestra, aplicando todas las etapas en cascada y actualizando automáticamente el estado interno.

Respetar esta secuencia —definición de coeficientes, asignación del buffer de estado, configuración de parámetros, inicialización y luego procesamiento— es fundamental para que el filtro funcione correctamente y se eviten errores comunes como desbordamientos o accesos inválidos a memoria.

Toda la documentación oficial puede consultarse en el sitio web de ARM:
<https://www.keil.com/pack/doc/CMSIS/DSP/html/index.html>.

3. Placa de Desarrollo y Herramientas Utilizadas

El proyecto fue desarrollado sobre la placa STM32 Nucleo-F446RE, basada en el microcontrolador STM32F446RE, perteneciente a la familia STM32F4 de STMicroelectronics. Esta plataforma integra una arquitectura avanzada ARM Cortex-M4, periféricos orientados a procesamiento de señales, y herramientas de desarrollo que permiten implementar sistemas en tiempo real de forma eficiente.

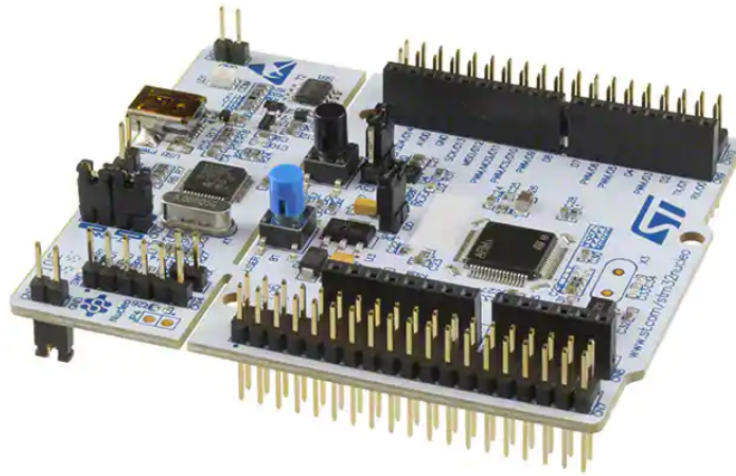


Figura 1: Placa de desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de 32 bits basado en el núcleo ARM Cortex-M4, optimizado para procesamiento digital con soporte de instrucciones DSP y unidad de punto flotante opcional (FPU simple precisión). Sus principales características relevantes al proyecto son:

- Frecuencia de operación: hasta 180MHz
- Memoria Flash: 512KB
- RAM: 128KB
- Tensión de operación: 1.8V a 3.6V
- Paquete: LQFP64 (compatible con la Nucleo-F446RE)

3.2. Periféricos clave utilizados

3.2.1. ADC (Analog-to-Digital Converter)

El STM32F446RE cuenta con 3 ADCs independientes (ADC1, ADC2 y ADC3), cada uno de 12 bits de resolución y con capacidad de hasta 2.4 MSPS (mega muestras por segundo) en modo intercalado. Para este proyecto se utiliza el ADC1 en modo regular y con DMA activado. Sus características técnicas relevantes incluyen:

- Resolución seleccionable: 6, 8, 10 o 12 bits (se usa 12 bits para mayor precisión).
- Tasa de muestreo máxima (single conversion): hasta 2.4 MSPS.
- Rango de entrada: 0V a VREF (típicamente 3.3V).
- Tiempo de muestreo configurable: desde 3 hasta 480 ciclos de ADC.
- Trigger externo: el inicio de la conversión puede ser disparado por un temporizador, lo cual permite muestrear a frecuencias precisas.
- Modo DMA: transferencia automática de los resultados a un buffer en memoria (sin intervención del CPU).
- Modo continuo + DMA circular: permite muestreo ininterrumpido con mínimo overhead.

3.2.2. DAC (Digital-to-Analog Converter)

El STM32F446RE incluye 2 canales DAC (DAC1 y DAC2), ambos de 12 bits, con capacidad de salida en modo bufferizado (con baja impedancia) o sin buffer (alta velocidad). Se utiliza DAC1 en este proyecto para convertir las muestras digitales de salida nuevamente en una señal analógica. Características clave:

- Resolución de conversión: 12, 10 u 8 bits.
- Velocidad típica: hasta 1 MSPS (modo sin buffer).
- Modo bufferizado: reduce el ruido, ideal para salidas de audio.
- Trigger externo: puede sincronizarse con un temporizador (por ejemplo TIM6/TRGO).
- Soporte DMA: se permite la alimentación directa de datos desde memoria al DAC sin CPU.
- Salidas disponibles: a través de pines analógicos dedicados (por ejemplo, PA4 para DAC1).

3.2.3. DMA (Direct Memory Access)

El controlador DMA2 es utilizado para las transferencias entre ADC/DAC y RAM. El STM32F446RE cuenta con dos controladores DMA (DMA1 y DMA2), con un total de 16 canales (streams) y soporte para múltiples periféricos. Características específicas:

- Transferencia en background, sin intervención del CPU.
- Modo circular: ideal para buffers cíclicos o doble buffering.

- Prioridad programable por canal.
- Transferencia por palabras (16 bits en Q15).
- Eventos por mitad de buffer y buffer completo: permite usar doble buffer sincronizado.

3.2.4. TIM (Timers)

El STM32F446RE cuenta con 14 temporizadores (4 avanzados, 2 básicos, 8 generales), todos con configuraciones de prescaler, auto-reload y posibilidad de generar eventos de actualización. Se utilizan para generar señales periódicas que actúan como trigger de muestreo. Características clave:

- TIM2-TIM5: de 32 bits, con amplio rango de conteo (útiles para precisión en frecuencias bajas).
- Configuración de frecuencia: vía Prescaler y ARR (Auto-Reload Register).
- Trigger de ADC/DAC: mediante señal TRGO (Trigger Output).
- Modo master-slave: permite sincronización entre periféricos.

3.2.5. GPIO – General Purpose Input/Output

Los pines GPIO permiten la interacción con el usuario mediante botones (teclas) y LEDs. Se utilizan:

- Velocidad de salida seleccionable: Low, Medium, Fast, High.
- Pull-up/Pull-down configurables.
- Interrupciones por flanco ascendente, descendente o ambos.

3.2.6. Herramientas de desarrollo

Para la implementación del proyecto se utilizaron las siguientes herramientas:

- STM32CubeIDE: entorno de desarrollo oficial de STMicroelectronics, basado en Eclipse, que permite configurar periféricos gráficamente, generar código y compilar con GCC.
- STM32CubeMX: herramienta para la configuración inicial de pines, relojes, periféricos y generación de código base.
- CMSIS y CMSIS-DSP: bibliotecas proporcionadas por ARM para facilitar el uso de instrucciones de bajo nivel y rutinas DSP optimizadas en punto fijo.
- GDB y ST-Link: utilizados para depuración, profiling y medición de tiempos mediante DWT->CYCCNT.

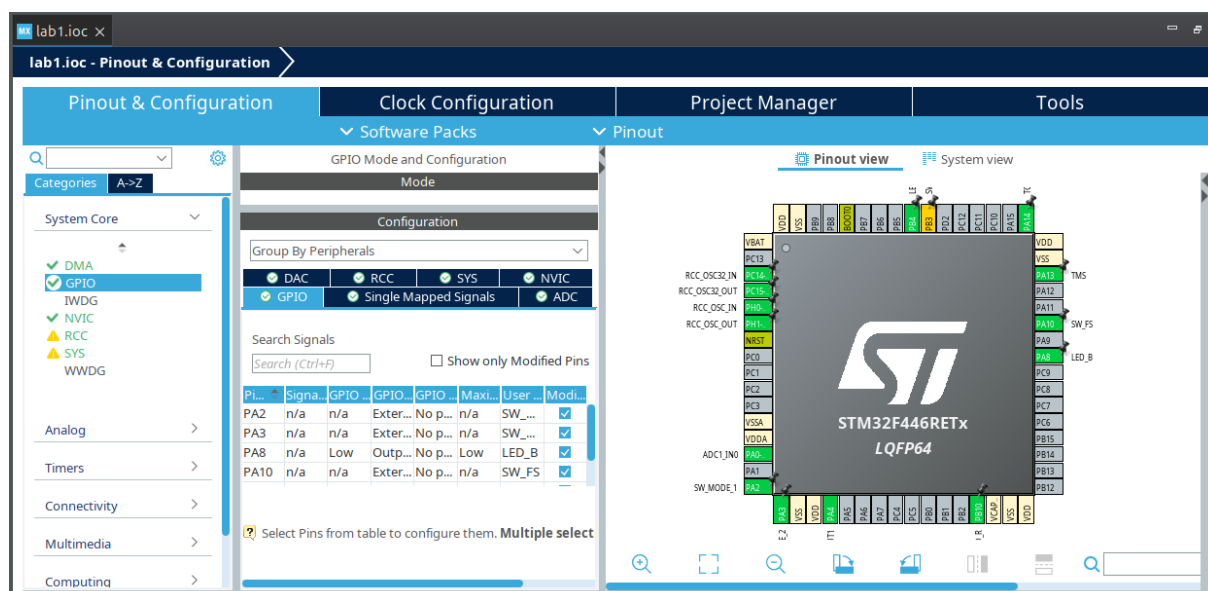


Figura 2: Entorno de desarrollo STM32CUBE IDE

4. Cálculo y diseño de filtros

El siguiente script en Python permite diseñar, visualizar y exportar filtros digitales IIR en formato compatible con la biblioteca CMSIS-DSP, especialmente para su uso en microcontroladores que trabajan con aritmética en punto fijo (Q15). Se definen varios tipos de filtros (pasa bajos, pasa altos, pasa banda y notch) y se evalúan sus respuestas en frecuencia para distintas frecuencias de muestreo. Cada filtro se diseña utilizando funciones de la biblioteca SciPy (`butter` para filtros clásicos y `iirnotch` para el notch), y luego se convierte a la forma de secciones en cascada (`tf2sos`) para garantizar estabilidad numérica. A partir de las secciones obtenidas, se calcula un valor de `postShift` sugerido que permite escalar correctamente los coeficientes al formato Q15 sin riesgo de saturación. Los coeficientes se imprimen en formato C, listos para ser copiados directamente en un proyecto basado en CMSIS-DSP, junto con los buffers de estado necesarios para cada configuración. Además, el script genera diagramas de Bode que permiten visualizar y comparar gráficamente el comportamiento en frecuencia de los filtros para distintas frecuencias de muestreo.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import butter, iirnotch, tf2sos, sosfreqz
4 # === Parametros generales ===
5 fs_list = [8000, 16000, 24000, 40000, 48000]
6 orden = 2
7 # === Especificaciones por tipo ===
8 filtros = {
9     "low": {"type": "lowpass", "params": {"fc": 3600}},
10    "high": {"type": "highpass", "params": {"fc": 35}},
11    "band": {"type": "bandpass", "params": {"fc1": 35, "fc2": 3500}},
12    "band_stop": {"type": "band_stop", "params": {"f0": 50, "bw": 30}}, #
13    Q = f0 / bw
14 }
15 # === Conversion a Q15 ===
16 def to_q15(x):
17     return max(min(round(x * 32767), 32767), -32768)
18 # === Loop por tipo de filtro y frecuencia de muestreo ===
19 for filt_name, spec in filtros.items():
20     plt.figure(figsize=(8, 5))
21     for fs in fs_list:

```

```

21     nyq = fs / 2
22     tipo = spec["type"]
23     params = spec["params"]
24     # ==== Diseño del filtro ====
25     if tipo == "lowpass":
26         Wn = params["fc"] / nyq
27         b, a = butter(orden, Wn, btype='low')
28     elif tipo == "highpass":
29         Wn = params["fc"] / nyq
30         b, a = butter(orden, Wn, btype='high')
31     elif tipo == "bandpass":
32         fc1 = params["fc1"] / nyq
33         fc2 = params["fc2"] / nyq
34         b, a = butter(orden, [fc1, fc2], btype='band')
35     elif tipo == "band_stop":
36         f0 = params["f0"]
37         Q = f0 / params["bw"]
38         w0 = f0 / nyq
39         b, a = iirnotch(w0=w0, Q=Q)
40     else:
41         continue
42     sos = tf2sos(b, a)
43     var_name = f"{filt_name}_{fs // 1000}k"
44     # ==== Calcular postShift sugerido ====
45     # Máximo coeficiente absoluto en punto flotante (solo numerador y denominador
46     # de cada sección)
47     max_coef = 0.0
48     for sec in sos:
49         b0, b1, b2, a0, a1, a2 = sec
50         max_sec = max(abs(b0), abs(b1), abs(b2), abs(a1), abs(a2))
51         if max_sec > max_coef:
52             max_coef = max_sec
53     # Escalado para evitar overflow en Q15 (coef entre -1 y 1 * 32767)
54     post_shift = max(0, int(np.floor(np.log2(32767 / max_coef))))
55     # ==== Generación de coeficientes Q15 ====
56     print(f"\n// ===== {tipo.upper()} Filter =====")
57     print(f"// fs = {fs} Hz")
58     print(f"// CMSIS-DSP format: {{b0, b1, b2, -a1, -a2}} in Q15")
59     print(f"// Suggested postShift: {post_shift}")
60     print(f"#define stage_{var_name} {len(sos)}")
61     print(f"q15_t {var_name}[5 * stage_{var_name}] = {{")
62         for sec in sos:
63             b0, b1, b2, a0, a1, a2 = sec
64             assert np.isclose(a0, 1.0, atol=1e-4), "a0 must be 1.0 for CMSIS-DSP"
65             b_q15 = [to_q15(c) for c in [b0, b1, b2]]
66             a_q15 = [to_q15(-a1), to_q15(-a2)]
67             line = "    " + ", ".join(f"{v:6d}" for v in (b_q15 + a_q15)) + ","
68             print(line)
69             print("};")
70     print(f"q15_t {var_name}_status[4 * stage_{var_name}] = {{ 0 }};\n")
71     # ==== Diagrama de Bode ====
72     w, h = sosfreqz(sos, worN=1024, fs=fs)
73     plt.semilogx(w, 20 * np.log10(np.maximum(np.abs(h), 1e-5)), label=f"{fs}
74     //1000}kHz")
75
76     # ==== Grafico por tipo de filtro ====
77     plt.title(f"Bode Plot - {tipo.title()} Filter")
78     plt.xlabel("Frequency [Hz]")
79     plt.ylabel("Magnitude [dB]")
80     plt.grid(which='both', linestyle='--', linewidth=0.5)
81     plt.axvline(params.get("fc", params.get("f0", params.get("fc1"))), color=
82     'red', linestyle=':', label="Reference Frequency")
83
84     if tipo == "bandpass":
85         plt.axvline(params["fc2"], color='red', linestyle=':')
86     elif tipo == "band_stop":
87         plt.axvline(params["f0"], color='red', linestyle=':')
88     plt.legend()
89     plt.tight_layout()
90     plt.show()

```

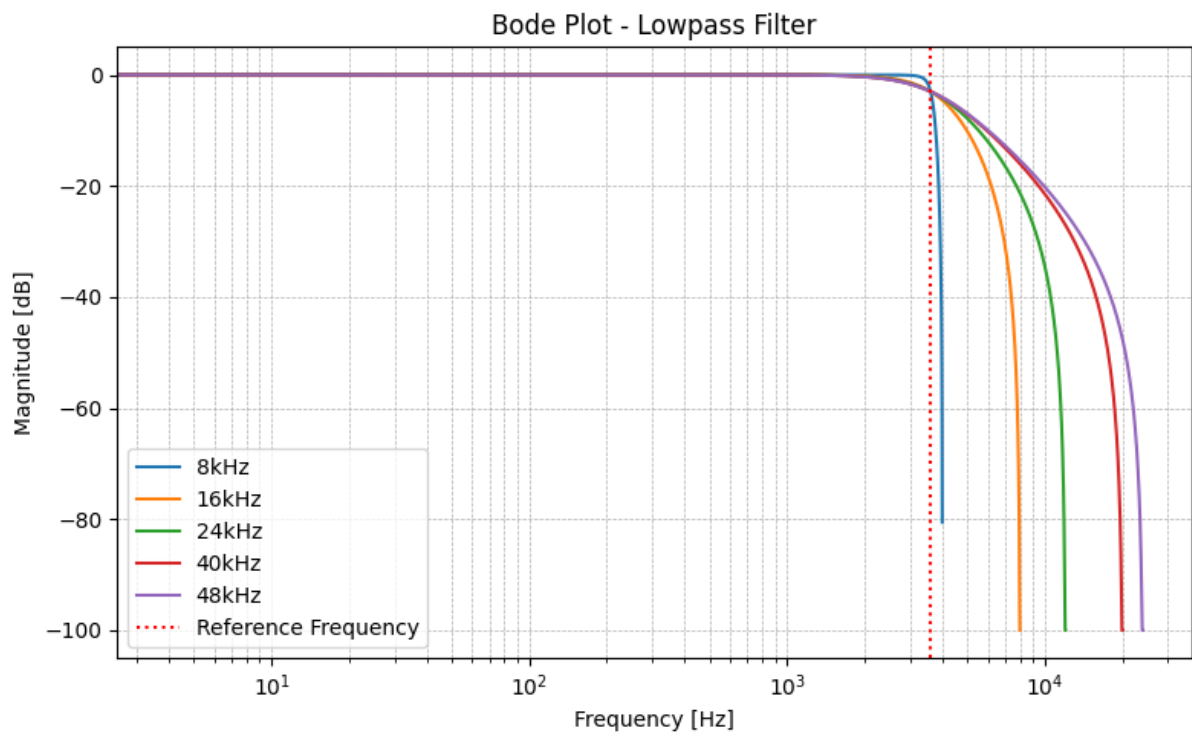


Figura 3: Diagrama de bode: Filtros pasa bajo

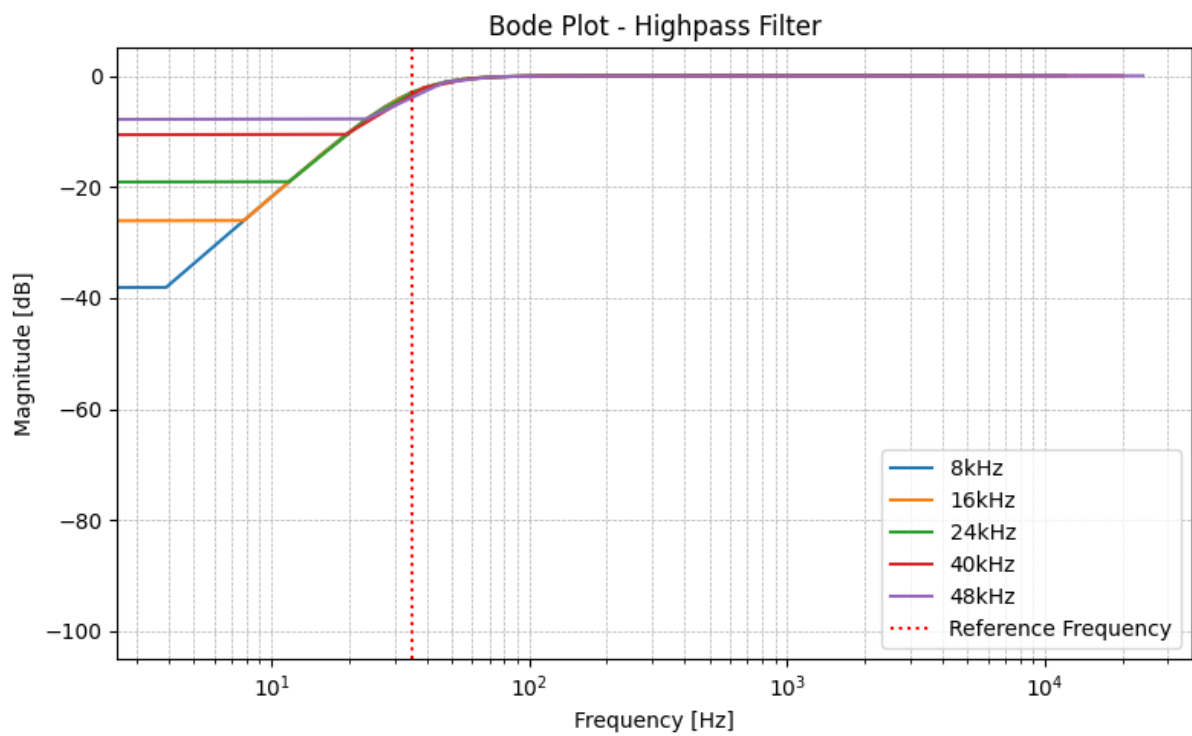


Figura 4: Diagrama de bode: Filtros pasa alto

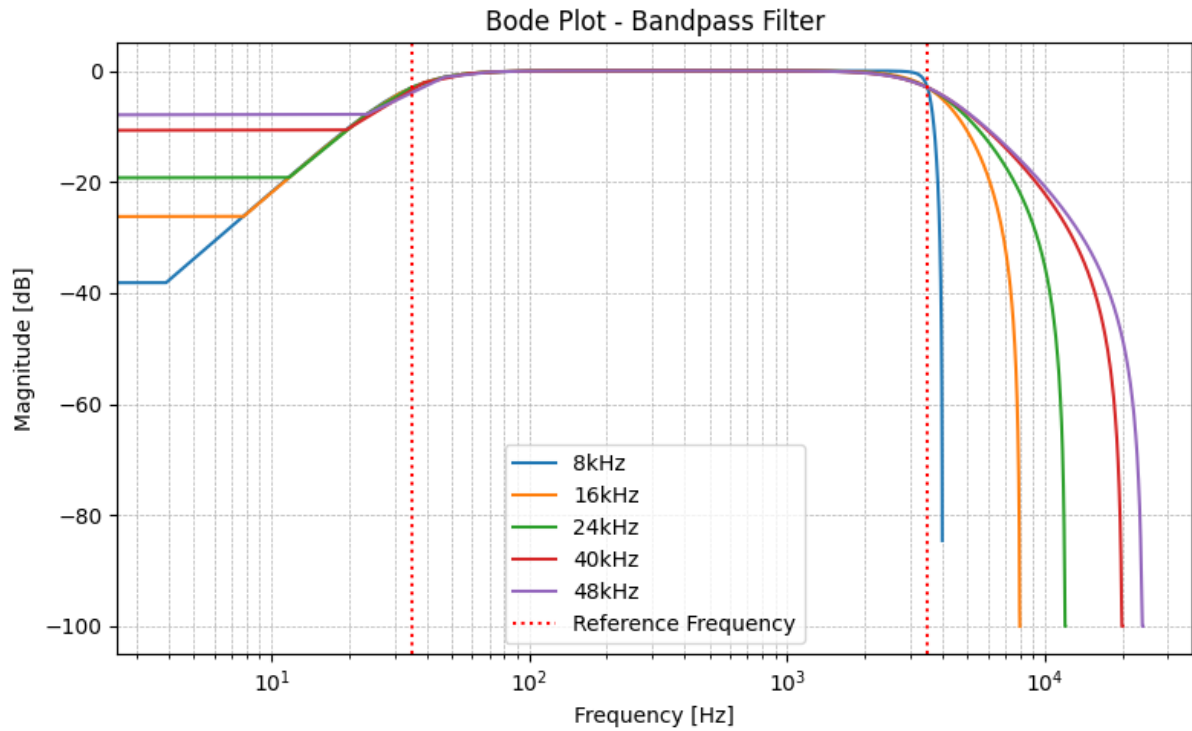


Figura 5: Diagrama de bode: Filtro pasa banda

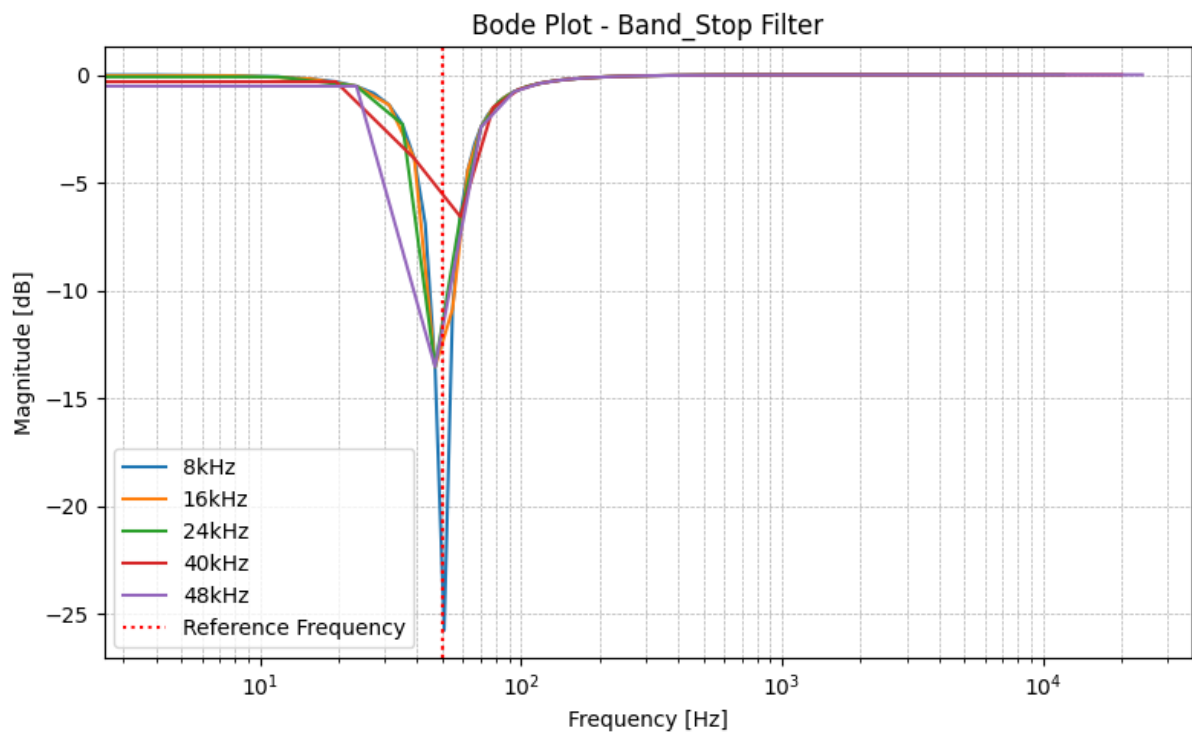


Figura 6: Diagrama de bode: Filtros rechaza banda

5. Diseño del programa

El sistema implementado permite procesar una señal en tiempo real utilizando el microcontrolador STM32F446RE. La estructura general es similar a la utilizada en el

Trabajo Práctico N^o1, incorporando en este caso una etapa de filtrado digital mediante filtros FIR implementados con la biblioteca CMSIS-DSP.

El funcionamiento del sistema comienza con la inicialización de los periféricos necesarios: GPIO, DMA, ADC, DAC y el temporizador TIM2. El temporizador define la frecuencia de muestreo y genera las señales de disparo para el ADC y el DAC, garantizando sincronización entre la adquisición y la reconstrucción de la señal.

La señal analógica de entrada es adquirida mediante el ADC con resolución de 12 bits. Las muestras se almacenan en memoria utilizando DMA en modo circular, lo que permite adquirir datos de forma continua sin intervención del CPU. El buffer de adquisición se divide en dos mitades, permitiendo procesar una mitad mientras la otra continúa llenándose con nuevas muestras.

Cuando el DMA completa la transferencia de una mitad del buffer, se ejecuta la rutina de procesamiento digital. En esta etapa, los datos se convierten al formato Q15 y se aplica el filtro seleccionado. El sistema permite elegir entre distintos tipos de filtros:

- filtro paso bajo
- filtro paso alto
- filtro pasa banda
- filtro rechaza banda

También es posible activar un modo bypass, en el cual la señal no es filtrada y se envía directamente a la salida.

Una vez procesadas, las muestras se almacenan en el buffer de salida, el cual es transferido mediante DMA hacia el DAC. El DAC reconstruye la señal analógica permitiendo observar el resultado del procesamiento en un osciloscopio.

La frecuencia de muestreo y el tipo de filtro pueden modificarse mediante pulsadores conectados a los pines GPIO. El sistema soporta frecuencias de muestreo de 8 kHz, 16 kHz, 22 kHz, 44 kHz y 48 kHz.

El procesamiento se realiza completamente en tiempo real utilizando interrupciones del DMA, sin necesidad de ejecutar código dentro del bucle principal del programa.

6. Programa principal

6.1. Implementación del main.c

```

1  /**
2  ****
3  * @file           : main.c
4  * @brief          : Real-Time Signal Acquisition and Filtering using STM32F446RE
5  *
6  * This program implements a real-time digital signal processing (DSP) pipeline
7  * on the STM32F446RE microcontroller. It captures analog input via ADC, processes
8  * the signal in Q15 fixed-point format using biquad IIR filters (LP, HP, BP, BS),
9  * and outputs the result through the DAC using DMA. The sampling rate (8 to 48 kHz)
10 * and filter type are dynamically selectable using external GPIO interrupts.
11 *
12 * Key Features:
13 * - Double-buffered ADC/DAC using DMA for real-time throughput
14 * - Q15 format filtering with CMSIS-DSP 'arm_biquad_cascade_df1_q15'
15 * - Dynamic selection of filter type and sampling frequency via GPIO
16 * - Processing time profiling using DWT->CYCCNT
17 * - Visual feedback using RGB LEDs
18 *
19 * Author           : Grupo DSP

```

```

20  * Date           : 24/06/2025
21  ****
22  */
23
24  /* USER CODE END Header */
25  /* Includes ----- */
26  #include "main.h"
27  #include "adc.h"
28  #include "dac.h"
29  #include "dma.h"
30  #include "tim.h"
31  #include "gpio.h"
32  #include "arm_math.h" // CMSIS-DSP library for fixed-point processing
33  #include "filtros.h"
34
35  /* USER CODE END Includes */
36
37  /* Private typedef ----- */
38  /* USER CODE BEGIN PTD */
39
40  /* USER CODE END PTD */
41
42  /* Private define ----- */
43  /* USER CODE BEGIN PD */
44  #define BUFFER_SIZE 2048 // Size of ADC/DAC/Q15 buffers
45  #define FS_8K 11249 // Auto-reload value for TIM2 to achieve ~8kHz
46  // sampling with 90 MHz clock
47  #define FS_16K 5624
48  #define FS_22K 4090
49  #define FS_44K 2041
50  #define FS_48K 1874
51  #define DEBOUNCE_DELAY 50 // Time anti-bounce
52
53  /* USER CODE END PD */
54
55  /* Private macro ----- */
56  /* USER CODE BEGIN PM */
57
58  /* USER CODE END PM */
59
60  /* Private variables ----- */
61  /* USER CODE BEGIN PV */
62  extern TIM_HandleTypeDef htim2; // Timer used for triggering ADC/DAC
63  extern ADC_HandleTypeDef hadc1; // ADC handle
64  extern DAC_HandleTypeDef hdac; // DAC handle
65
66  uint16_t adc_buffer[BUFFER_SIZE]; // Raw ADC samples
67  uint16_t dac_buffer[BUFFER_SIZE]; // Output buffer for DAC
68  q15_t q15_buffer[BUFFER_SIZE / 2]; // Intermediate Q15 format buffer
69  q15_t q15_buffer_2[BUFFER_SIZE / 2];
70  uint8_t fs_mode = 0; // Sampling frequency mode selector (0 to 5)
71  uint32_t time = 0; // Variable to store CPU cycles for processing time profiling
72  uint8_t filter_type = 0; // type of the filter LP,HP,BP,BS
73  uint32_t last_interrupt_time = 0; // Time of the last interrupt anti-bouncing
74  uint8_t flag = 0;
75
76  q15_t FPB_status[FPB_48K_NUM_TAPS + BUFFER_SIZE / 2];
77  q15_t FPA_status[FPA_48K_NUM_TAPS + BUFFER_SIZE / 2];
78  q15_t FPBD_status[FPBD_48K_NUM_TAPS + BUFFER_SIZE / 2];
79  q15_t FRBD_status[FRBD_48K_NUM_TAPS + BUFFER_SIZE / 2];
80
81  arm_fir_instance_q15 FPB;
82  arm_fir_instance_q15 FPA;
83  arm_fir_instance_q15 FPBD;
84  arm_fir_instance_q15 FRBD;
85
86  /* USER CODE END PV */
87
88  /* Private function prototypes ----- */
89  void SystemClock_Config(void);
90  /* USER CODE BEGIN PFP */
91
92  /* USER CODE END PFP */
93
94  /* Private user code ----- */

```

```

95  /* USER CODE BEGIN 0 */
96
97  /**
98   * @brief Converts a Q15 array to a 12-bit DAC buffer
99   * @param in: Input Q15 data array
100  * @param out: Output DAC buffer
101  * @param len: Number of elements to convert
102  */
103  void convert_q15_to_dac_buffer(q15_t *in, uint16_t *out, uint16_t len) {
104      for (uint16_t i = 0; i < len; i++) {
105          int32_t val = ((int32_t) in[i] * 4095) / 32767;
106          if (val < 0)
107              val = 0;
108          else if (val > 4095)
109              val = 4095;
110          out[i] = (uint16_t) val;
111      }
112  }
113
114  /**
115   * @brief Processes half of the ADC buffer: converts to Q15, applies processing, and
116   *         prepares DAC output
117   * @param adc_in: Input ADC buffer
118   * @param q15_out: Intermediate Q15 buffer
119   * @param dac_out: Output DAC buffer
120   * @param offset: Offset to select first or second half of buffer
121  */
122  void process_buffer_half(uint16_t *adc_in, uint16_t *dac_out, uint16_t offset) {
123      uint32_t start = DWT->CYCCNT; // Start cycle counter for performance profiling
124
125      // Convert ADC 12-bit values to Q15 format
126      for (uint16_t i = 0; i < BUFFER_SIZE / 2; i++) {
127          q15_buffer[i] = (q15_t) (((int32_t) adc_in[i + offset] * 32767) / 4095);
128      }
129
130      if (flag == 0) {
131          switch (filter_type) {
132              case 0:
133                  arm_fir_q15(&FPB, &q15_buffer[0], &q15_buffer_2[0],
134                           BUFFER_SIZE / 2);
135                  break;
136              case 1:
137                  arm_fir_q15(&FPA, &q15_buffer[0], &q15_buffer_2[0],
138                           BUFFER_SIZE / 2);
139                  break;
140              case 2:
141                  arm_fir_q15(&FPBD, &q15_buffer[0], &q15_buffer_2[0],
142                           BUFFER_SIZE / 2);
143                  break;
144              case 3:
145                  arm_fir_q15(&FRBD, &q15_buffer[0], &q15_buffer_2[0],
146                           BUFFER_SIZE / 2);
147                  break;
148              default:
149                  break;
150          }
151
152          // Convert processed Q15 data to DAC output format
153          convert_q15_to_dac_buffer(&q15_buffer_2[0], &dac_out[offset],
154                                   BUFFER_SIZE / 2);
155
156          time = DWT->CYCCNT - start; // Measure processing time in CPU cycles
157      }
158
159  /**
160   * @brief DMA Half Transfer Complete Callback for ADC
161  */
162  void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
163      if (hadc->Instance == ADC1) {
164          process_buffer_half(adc_buffer, dac_buffer, 0);
165      }
166  }
167
168  /**
169

```

```

170  * @brief DMA Transfer Complete Callback for ADC
171  */
172  void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
173      if (hadc->Instance == ADC1) {
174          process_buffer_half(adc_buffer, dac_buffer,
175                          BUFFER_SIZE / 2);
176      }
177  }
178  }
179
180  /**
181  * @brief GPIO interrupt callback for switching sampling frequencies or stopping
182  *       acquisition
183  *       Cycles through 6 modes on button press (pin: SW_FS) and switching type
184  *       filter through
185  *       4 modes, BP, HP, BP and BS on button press (pin: SW_MODE_1)
186  */
187  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
188
189      uint32_t current_time = HAL_GetTick();
190      if ((current_time - last_interrupt_time) < DEBOUNCE_DELAY) {
191          return;
192      }
193      last_interrupt_time = current_time;
194
195      // Stop timer before reconfiguring
196      __HAL_TIM_DISABLE(&htim2);
197
198      // Clean all buffers status
199      arm_fill_q15(0, &FPB_status, FPB_48K_NUM_TAPS + BUFFER_SIZE / 2);
200      arm_fill_q15(0, &FPA_status, FPA_48K_NUM_TAPS + BUFFER_SIZE / 2);
201      arm_fill_q15(0, &FPBD_status, FPBD_48K_NUM_TAPS + BUFFER_SIZE / 2);
202      arm_fill_q15(0, &FRBD_status, FRBD_48K_NUM_TAPS + BUFFER_SIZE / 2);
203
204      arm_fill_q15(0, &adc_buffer, BUFFER_SIZE);
205      arm_fill_q15(0, &dac_buffer, BUFFER_SIZE);
206      arm_fill_q15(0, &q15_buffer, BUFFER_SIZE / 2);
207      arm_fill_q15(0, &q15_buffer_2, BUFFER_SIZE / 2);
208
209      if (GPIO_Pin == SW_FS_Pin) {
210          fs_mode = (fs_mode + 1) % 6;
211
212          switch (fs_mode) {
213              case 0:
214                  __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K); // 8 kHz
215                  HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_SET); // Blue
216                  HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);
217                  HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
218
219                  arm_fir_init_q15(&FPB, FPB_8K_NUM_TAPS, FPB_8K_COEFFS, FPB_status,
220                          BUFFER_SIZE / 2);
221                  arm_fir_init_q15(&FPA, FPA_8K_NUM_TAPS, FPA_8K_COEFFS, FPA_status,
222                          BUFFER_SIZE / 2);
223                  arm_fir_init_q15(&FPBD, FPBD_8K_NUM_TAPS, FPBD_8K_COEFFS,
224                          FPBD_status,
225                          BUFFER_SIZE / 2);
226                  arm_fir_init_q15(&FRBD, FRBD_8K_NUM_TAPS, FRBD_8K_COEFFS,
227                          FRBD_status,
228                          BUFFER_SIZE / 2);
229                  break;
230              case 1:
231                  __HAL_TIM_SET_AUTORELOAD(&htim2, FS_16K); // ~16 kHz
232                  HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_RESET);
233                  HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_SET); // Red
234                  HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
235
236                  arm_fir_init_q15(&FPB, FPB_16K_NUM_TAPS, FPB_16K_COEFFS, FPB_status,
237                          BUFFER_SIZE / 2);
238                  arm_fir_init_q15(&FPA, FPA_16K_NUM_TAPS, FPA_16K_COEFFS, FPA_status,
239                          BUFFER_SIZE / 2);
240                  arm_fir_init_q15(&FPBD, FPBD_16K_NUM_TAPS, FPBD_16K_COEFFS,
241                          FPBD_status,
242                          BUFFER_SIZE / 2);
243                  arm_fir_init_q15(&FRBD, FRBD_16K_NUM_TAPS, FRBD_16K_COEFFS,
244                          FRBD_status,
245                          BUFFER_SIZE / 2);

```

```

244     break;
245 case 2:
246     __HAL_TIM_SET_AUTORELOAD(&htim2, FS_22K); // ~22 kHz
247     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_RESET);
248     HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);
249     HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_SET); // Green
250
251     arm_fir_init_q15(&FPB, FPB_22K_NUM_TAPS, FPB_22K_COEFFS, FPB_status,
252     BUFFER_SIZE / 2);
253     arm_fir_init_q15(&FPA, FPA_22K_NUM_TAPS, FPA_22K_COEFFS, FPA_status,
254     BUFFER_SIZE / 2);
255     arm_fir_init_q15(&FPBD, FPBD_22K_NUM_TAPS, FPBD_22K_COEFFS,
256     FPBD_status,
257     BUFFER_SIZE / 2);
258     arm_fir_init_q15(&FRBD, FRBD_22K_NUM_TAPS, FRBD_22K_COEFFS,
259     FRBD_status,
260     BUFFER_SIZE / 2);
261     break;
262 case 3:
263     __HAL_TIM_SET_AUTORELOAD(&htim2, FS_44K); // ~44 kHz
264     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_RESET);
265     HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_SET); // Red + Green =
266     Yellow
267     HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_SET);
268
269     arm_fir_init_q15(&FPB, FPB_44K_NUM_TAPS, FPB_44K_COEFFS, FPB_status,
270     BUFFER_SIZE / 2);
271     arm_fir_init_q15(&FPA, FPA_44K_NUM_TAPS, FPA_44K_COEFFS, FPA_status,
272     BUFFER_SIZE / 2);
273     arm_fir_init_q15(&FPBD, FPBD_44K_NUM_TAPS, FPBD_44K_COEFFS,
274     FPBD_status,
275     BUFFER_SIZE / 2);
276     arm_fir_init_q15(&FRBD, FRBD_44K_NUM_TAPS, FRBD_44K_COEFFS,
277     FRBD_status,
278     BUFFER_SIZE / 2);
279     break;
280 case 4:
281     __HAL_TIM_SET_AUTORELOAD(&htim2, FS_48K); // ~48 kHz
282     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_SET); // Blue + Red =
283     Purple
284     HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_SET);
285     HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
286
287     arm_fir_init_q15(&FPB, FPB_48K_NUM_TAPS, FPB_48K_COEFFS, FPB_status,
288     BUFFER_SIZE / 2);
289     arm_fir_init_q15(&FPA, FPA_48K_NUM_TAPS, FPA_48K_COEFFS, FPA_status,
290     BUFFER_SIZE / 2);
291     arm_fir_init_q15(&FPBD, FPBD_48K_NUM_TAPS, FPBD_48K_COEFFS,
292     FPBD_status,
293     BUFFER_SIZE / 2);
294     arm_fir_init_q15(&FRBD, FRBD_48K_NUM_TAPS, FRBD_48K_COEFFS,
295     FRBD_status,
296     BUFFER_SIZE / 2);
297     break;
298 case 5:
299     // Stop all activity
300     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_RESET); // All off
301     HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);
302     HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
303     flag = !flag;
304
305     return;
306 }
307 }
308
309 if (GPIO_Pin == SW_MODE_1_Pin) {
310     filter_type = (filter_type + 1) % 4;
311 }
312
313 // Reset and enable timer
314 __HAL_TIM_SET_COUNTER(&htim2, 0);
315 TIM2->EGR = TIM_EGR_UG;
316 __HAL_TIM_ENABLE(&htim2);
317 }

```

```

318  /* USER CODE END 0 */
319
320  /**
321   * @brief The application entry point.
322   * @retval int
323   */
324  int main(void) {
325
326      /* USER CODE BEGIN 1 */
327      // Enable DWT cycle counter for profiling purposes
328      CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
329      DWT->CYCCNT = 0;
330      DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
331      /* USER CODE END 1 */
332
333      /* MCU Configuration ----- */
334
335      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
336      HAL_Init();
337
338      /* USER CODE BEGIN Init */
339
340      /* USER CODE END Init */
341
342      /* Configure the system clock */
343      SystemClock_Config();
344
345      /* USER CODE BEGIN SysInit */
346
347      /* USER CODE END SysInit */
348
349      /* Initialize all configured peripherals */
350      MX_GPIO_Init();
351      MX_DMA_Init();
352      MX_ADC1_Init();
353      MX_TIM2_Init();
354      MX_DAC_Init();
355      /* USER CODE BEGIN 2 */
356
357      HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_SET); // Blue
358      HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);
359      HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
360
361      // Start base timer
362      HAL_TIM_Base_Start(&htim2);
363
364      // Start DAC in DMA mode
365      HAL_DAC_Start_DMA(&hdac, DAC_CHANNEL_1, (uint32_t*) dac_buffer,
366      BUFFER_SIZE, DAC_ALIGN_12B_R);
367
368      // Start ADC in DMA mode
369      HAL_ADC_Start_DMA(&hadc1, (uint32_t*) adc_buffer, BUFFER_SIZE);
370
371      arm_fir_init_q15(&FPB, FPB_8K_NUM_TAPS, FPB_8K_COEFFS, FPB_status,
372      BUFFER_SIZE / 2);
373      arm_fir_init_q15(&FPA, FPA_8K_NUM_TAPS, FPA_8K_COEFFS, FPA_status,
374      BUFFER_SIZE / 2);
375      arm_fir_init_q15(&FPBD, FPBD_8K_NUM_TAPS, FPBD_8K_COEFFS, FPBD_status,
376      BUFFER_SIZE / 2);
377      arm_fir_init_q15(&FRBD, FRBD_8K_NUM_TAPS, FRBD_8K_COEFFS, FRBD_status,
378      BUFFER_SIZE / 2);
379
380      /* USER CODE END 2 */
381
382      /* Infinite loop */
383      /* USER CODE BEGIN WHILE */
384      while (1) {
385          /* USER CODE END WHILE */
386
387          /* USER CODE BEGIN 3 */
388          // Main loop intentionally left empty. All processing handled via DMA callbacks
389          // and interrupts.
390      }
391      /* USER CODE END 3 */
392  }

```

```

393  /**
394  * @brief System Clock Configuration
395  * @retval None
396  */
397  void SystemClock_Config(void) {
398      RCC_OscInitTypeDef RCC_OscInitStruct = { 0 };
399      RCC_ClkInitTypeDef RCC_ClkInitStruct = { 0 };
400
401      /** Configure the main internal regulator output voltage
402      */
403      __HAL_RCC_PWR_CLK_ENABLE();
404      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
405
406      /** Initializes the RCC Oscillators according to the specified parameters
407      * in the RCC_OscInitTypeDef structure.
408      */
409      RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
410      RCC_OscInitStruct.HSISState = RCC_HSI_ON;
411      RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
412      RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
413      RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
414      RCC_OscInitStruct.PLL.PLLM = 8;
415      RCC_OscInitStruct.PLL.PLLN = 180;
416      RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
417      RCC_OscInitStruct.PLL.PLLQ = 2;
418      RCC_OscInitStruct.PLL.PLLR = 2;
419      if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
420          Error_Handler();
421      }
422
423      /** Activate the Over-Drive mode
424      */
425      if (HAL_PWREx_EnableOverDrive() != HAL_OK) {
426          Error_Handler();
427      }
428
429      /** Initializes the CPU, AHB and APB buses clocks
430      */
431      RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
432          | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
433      RCC_ClkInitStruct.SYSClkSource = RCC_SYSCCLKSOURCE_PLLCLK;
434      RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCCLK_DIV1;
435      RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
436      RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
437
438      if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK) {
439          Error_Handler();
440      }
441  }
442
443  /* USER CODE BEGIN 4 */
444
445  /* USER CODE END 4 */
446
447  /**
448  * @brief This function is executed in case of error occurrence.
449  * @retval None
450  */
451  void Error_Handler(void) {
452      /* USER CODE BEGIN Error_Handler_Debug */
453      // Stay in infinite loop on error
454      __disable_irq();
455      while (1) {
456      }
457      /* USER CODE END Error_Handler_Debug */
458  }
459
460  #ifndef USE_FULL_ASSERT
461  /**
462  * @brief Reports the name of the source file and the source line number
463  * where the assert_param error has occurred.
464  * @param file: pointer to the source file name
465  * @param line: assert_param error line source number
466  * @retval None
467  */
468  void assert_failed(uint8_t *file, uint32_t line)

```

```
469 {  
470  /* USER CODE BEGIN 6 */  
471  // Custom assert handler (optional user-defined implementation)  
472  /* USER CODE END 6 */  
473 }  
474 #endif /* USE_FULL_ASSERT */
```

6.2. Configuración de periféricos

El programa principal inicializa los periféricos necesarios para realizar la adquisición, procesamiento y reproducción de la señal en tiempo real. La arquitectura general mantiene el mismo esquema del Trabajo Práctico N^o1, incorporando ahora la etapa de filtrado digital mediante la biblioteca CMSIS-DSP.

La inicialización de los periféricos se realiza mediante las funciones generadas automáticamente por STM32CubeMX:

- MX_GPIO_Init()
- MX_DMA_Init()
- MX_ADC1_Init()
- MX_TIM2_Init()
- MX_DAC_Init()

Cada uno de estos periféricos cumple una función específica dentro del sistema de procesamiento digital en tiempo real.

6.2.1. Configuración del ADC

El conversor analógico-digital (ADC1) se utiliza para adquirir la señal de entrada con una resolución de 12 bits. El ADC es disparado periódicamente por el temporizador TIM2, lo que garantiza una frecuencia de muestreo precisa.

La transferencia de datos se realiza mediante DMA en modo circular, permitiendo almacenar continuamente las muestras en un buffer sin intervención directa del CPU.

La inicialización del ADC en modo DMA se realiza con:

```
1  HAL_ADC_Start_DMA(&hadc1,  
2      (uint32_t*) adc_buffer,  
3      BUFFER_SIZE);
```

El uso de DMA permite implementar un esquema de doble buffer, donde la mitad del buffer puede procesarse mientras la otra mitad continúa llenándose con nuevas muestras.

6.2.2. Configuración del DAC

El conversor digital-analógico (DAC1) se utiliza para reconstruir la señal procesada y permitir su visualización mediante un osciloscopio.

El DAC también opera en modo DMA, lo que permite enviar continuamente los datos procesados desde memoria hacia la salida analógica sin intervención del CPU.

La inicialización del DAC se realiza mediante:

```
1  HAL_DAC_Start_DMA(&hdac ,  
2      DAC_CHANNEL_1 ,  
3      (uint32_t*) dac_buffer ,  
4      BUFFER_SIZE ,  
5      DAC_ALIGN_12B_R);
```

6.2.3. Configuración del temporizador

El temporizador TIM2 se utiliza como base de tiempo para definir la frecuencia de muestreo del sistema. Este temporizador genera eventos periódicos (TRGO) que disparan simultáneamente el ADC y el DAC, garantizando sincronización entre adquisición y reconstrucción de la señal.

La frecuencia de muestreo puede modificarse dinámicamente mediante interrupciones externas, ajustando el valor del registro Auto-Reload del temporizador.

El temporizador se inicia mediante:

```
1  HAL_TIM_Base_Start(&htim2);
```

Los valores de autoreload utilizados para cada frecuencia de muestreo son:

- 8 kHz
- 16 kHz
- 22 kHz
- 44 kHz
- 48 kHz

6.2.4. Configuración del DMA

El controlador DMA permite transferir datos entre los periféricos y la memoria sin intervención del CPU. En este sistema se utiliza para:

- Transferir muestras desde el ADC hacia el buffer de entrada
- Transferir muestras procesadas hacia el DAC

El uso de DMA reduce la carga del procesador y permite garantizar procesamiento en tiempo real.

6.2.5. Configuración de GPIO

Los pines GPIO se utilizan para interactuar con el usuario mediante pulsadores y LEDs.

Las interrupciones externas permiten:

- Cambiar la frecuencia de muestreo
- Cambiar el tipo de filtro (LP, HP, BP, BS)
- Activar el modo bypass del filtro

Además, los LEDs RGB indican visualmente el estado del sistema y la frecuencia de muestreo seleccionada.

6.2.6. Inicialización de los filtros digitales

Luego de inicializar los periféricos, se configuran las estructuras necesarias para implementar los filtros digitales en formato Q15 utilizando la biblioteca CMSIS-DSP.

La inicialización de los filtros se realiza mediante:

```
1  arm_fir_init_q15(&FPB,
2      FPB_8K_NUM_TAPS,
3      FPB_8K_COEFFS,
4      FPB_status,
5      BUFFER_SIZE / 2);
```

Se definen filtros paso bajo, paso alto, pasa banda y rechazo de banda para cada frecuencia de muestreo soportada por el sistema.

6.2.7. Ejecución en tiempo real

Una vez inicializados los periféricos, el sistema comienza a operar en tiempo real mediante interrupciones del DMA. El procesamiento de la señal se realiza dentro de las funciones callback:

- HAL_ADC_ConvHalfCpltCallback()
- HAL_ADC_ConvCpltCallback()

Estas funciones procesan cada mitad del buffer utilizando los filtros digitales seleccionados.

7. Resultados experimentales

7.1. Filtro pasa bajo

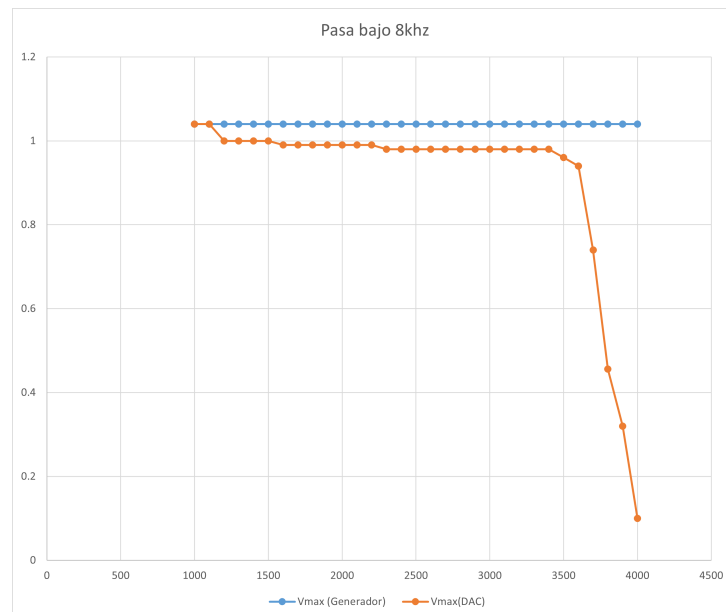


Figura 7: Respuesta en amplitud del filtro pasa bajo para una frecuencia de muestreo de 8 kHz.

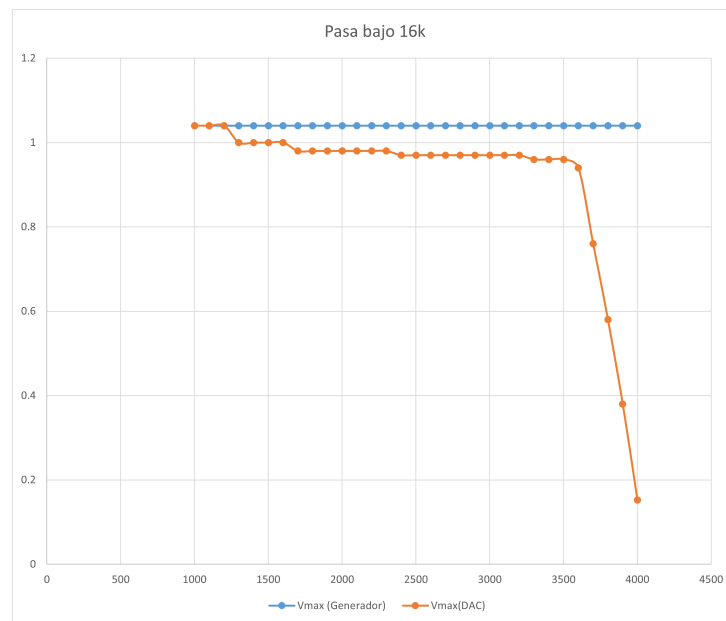


Figura 8: Respuesta en amplitud del filtro pasa bajo para una frecuencia de muestreo de 16 kHz.

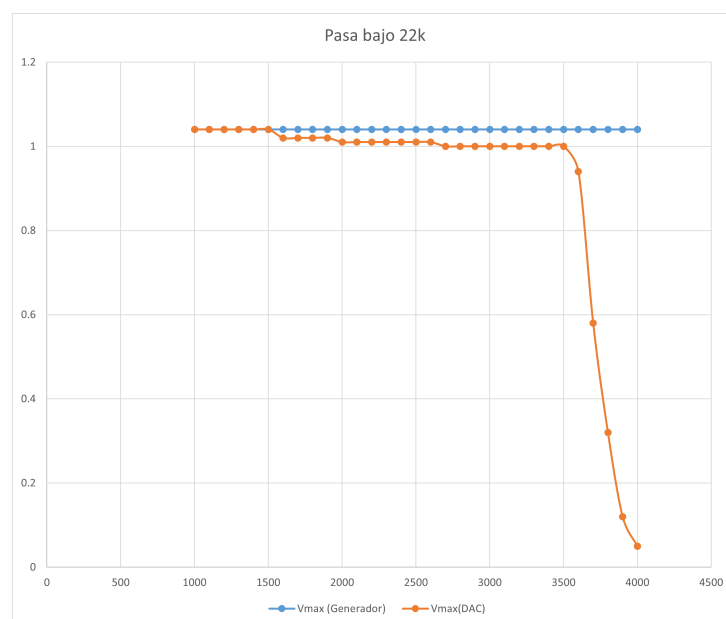


Figura 9: Respuesta en amplitud del filtro pasa bajo para una frecuencia de muestreo de 22 kHz.

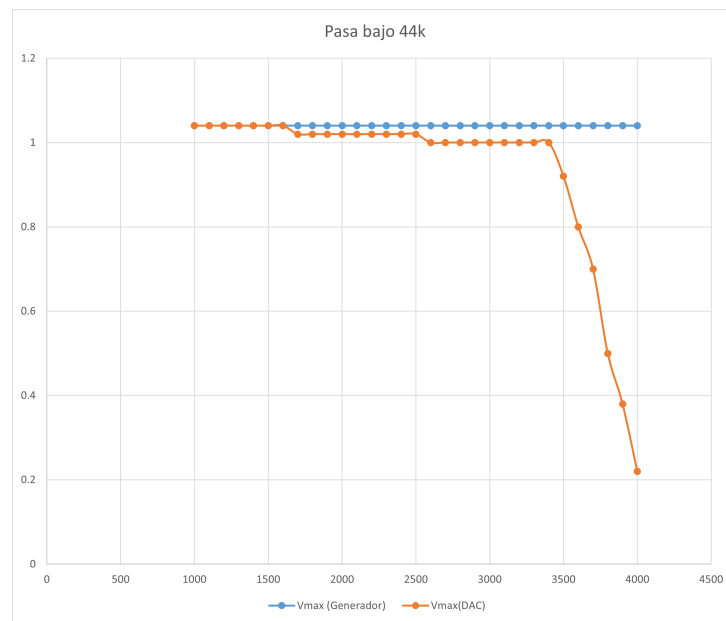


Figura 10: Respuesta en amplitud del filtro pasa bajo para una frecuencia de muestreo de 44 kHz.

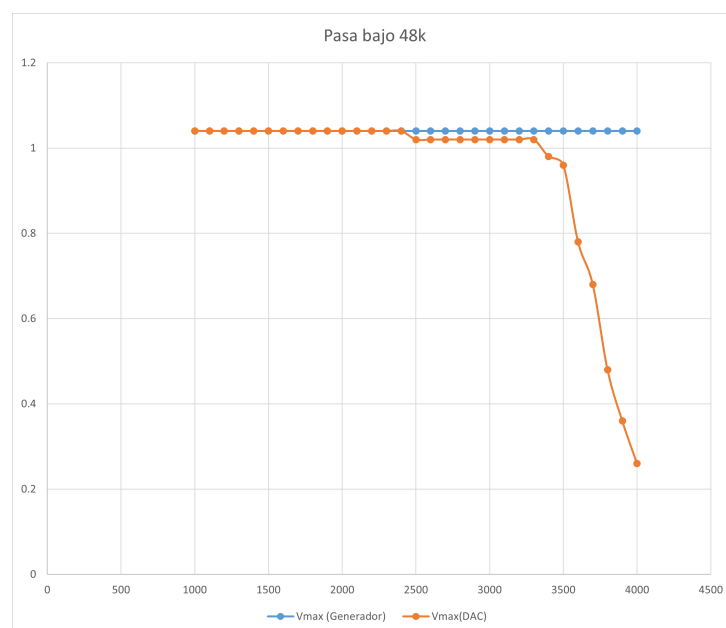


Figura 11: Respuesta en amplitud del filtro pasa bajo para una frecuencia de muestreo de 48 kHz.

7.2. Filtro pasa alto

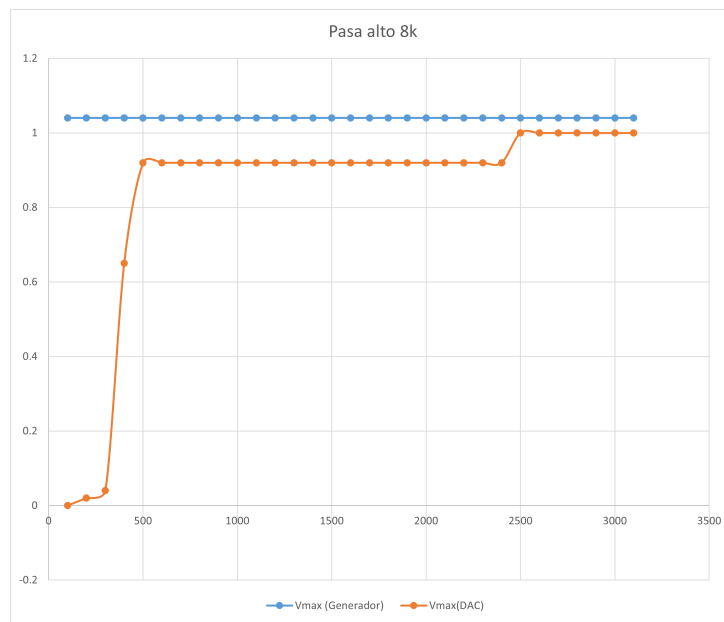


Figura 12: Respuesta en amplitud del filtro pasa alto para una frecuencia de muestreo de 8 kHz.

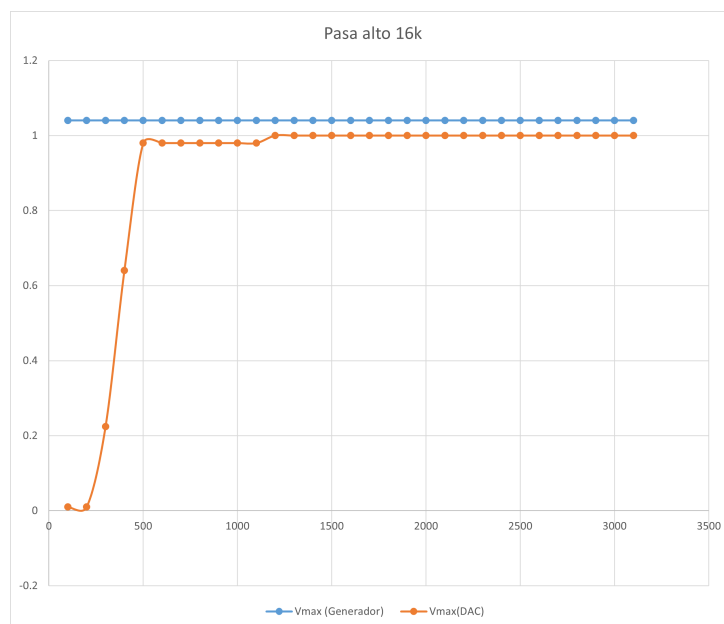


Figura 13: Respuesta en amplitud del filtro pasa alto para una frecuencia de muestreo de 16 kHz.

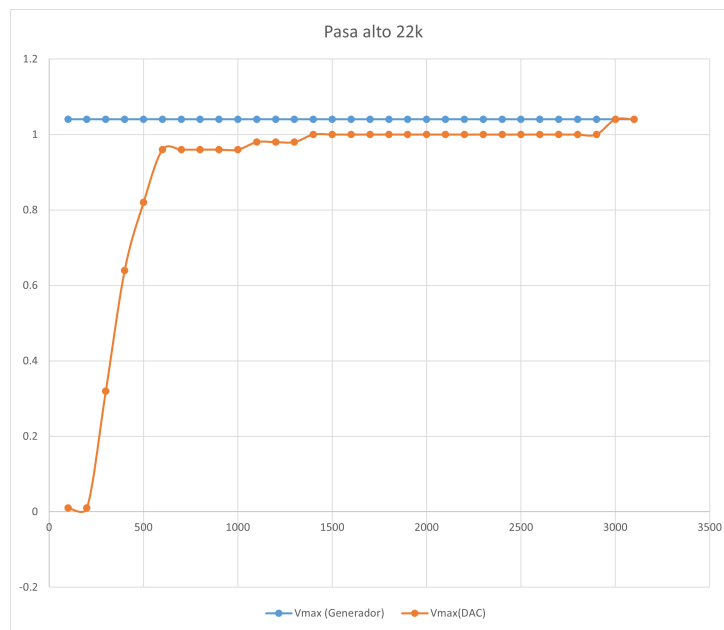


Figura 14: Respuesta en amplitud del filtro pasa alto para una frecuencia de muestreo de 22 kHz.

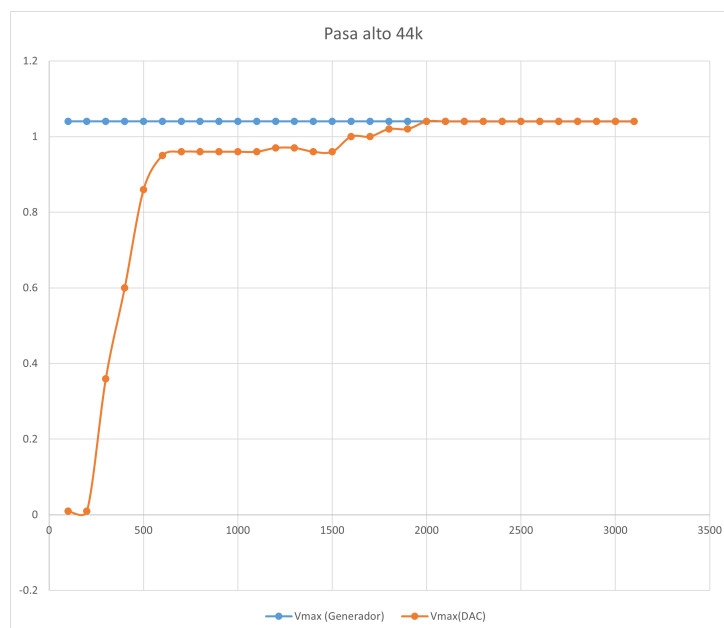


Figura 15: Respuesta en amplitud del filtro pasa alto para una frecuencia de muestreo de 44 kHz.

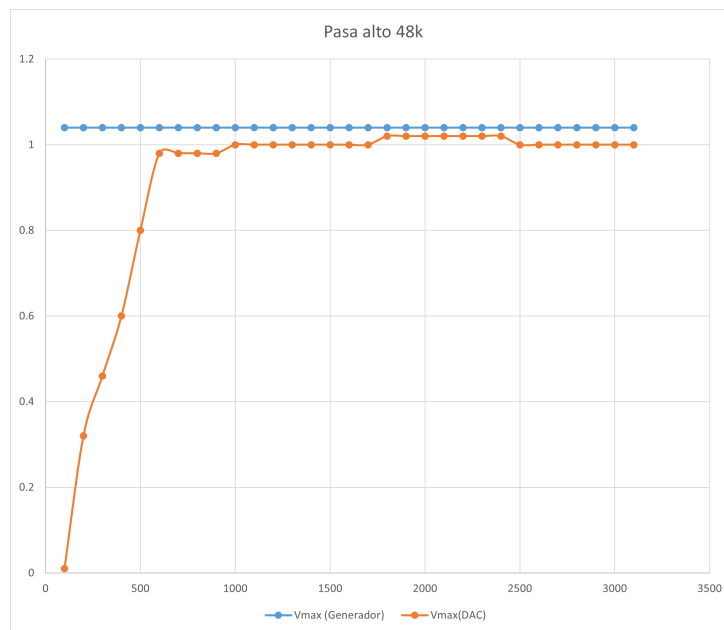


Figura 16: Respuesta en amplitud del filtro pasa alto para una frecuencia de muestreo de 48 kHz.

7.3. Filtro pasa banda

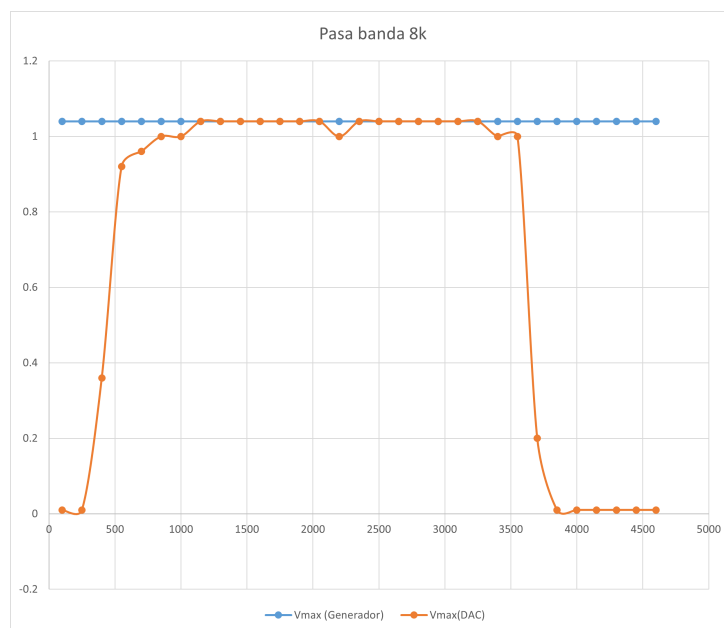


Figura 17: Respuesta en amplitud del filtro pasa banda para una frecuencia de muestreo de 8 kHz.

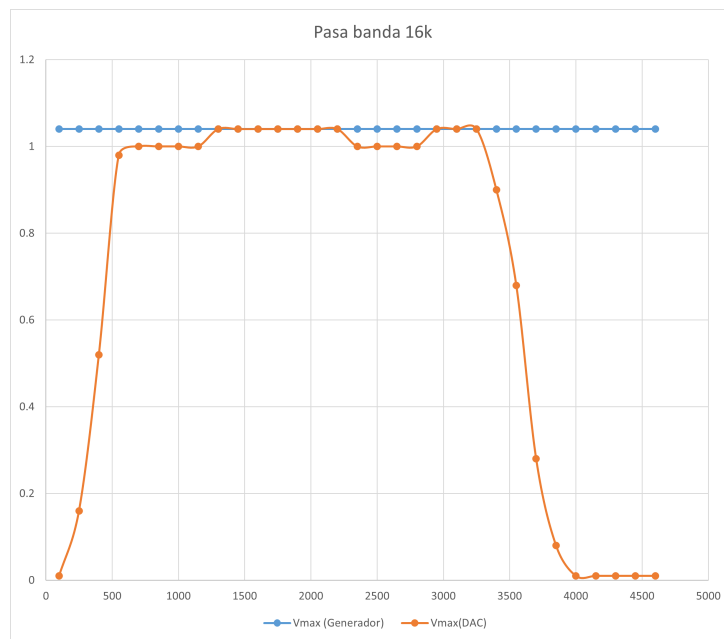


Figura 18: Respuesta en amplitud del filtro pasa banda para una frecuencia de muestreo de 16 kHz.

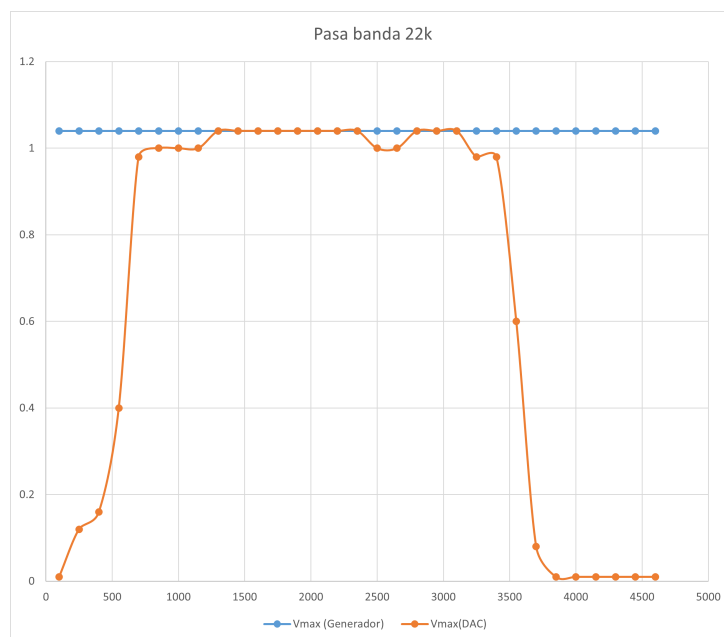


Figura 19: Respuesta en amplitud del filtro pasa banda para una frecuencia de muestreo de 22 kHz.

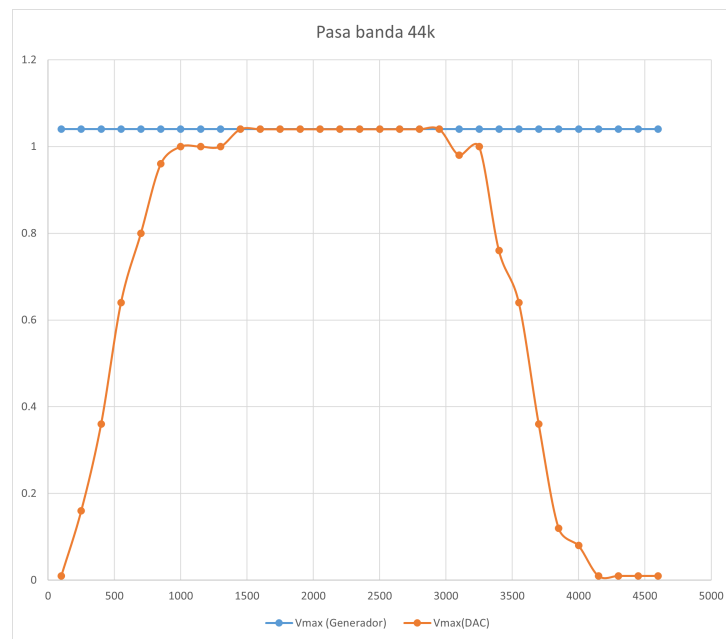


Figura 20: Respuesta en amplitud del filtro pasa banda para una frecuencia de muestreo de 44 kHz.

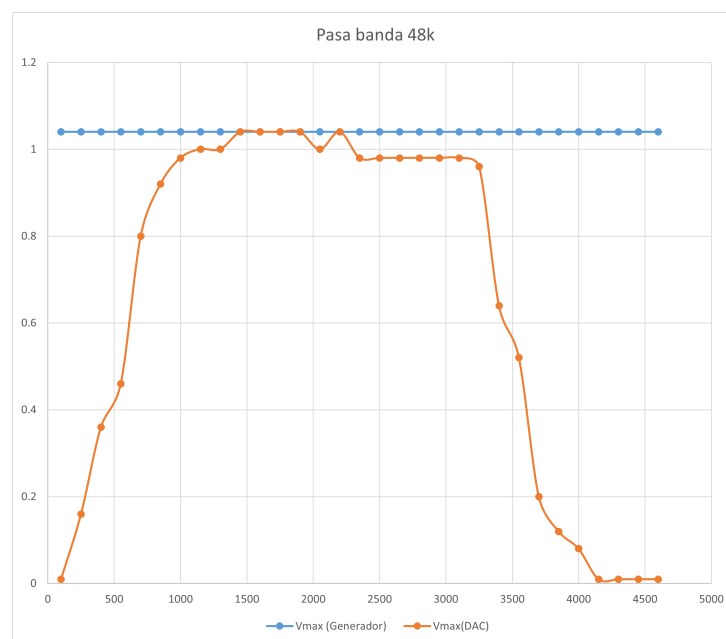


Figura 21: Respuesta en amplitud del filtro pasa banda para una frecuencia de muestreo de 48 kHz.

7.4. Filtro rechaza banda

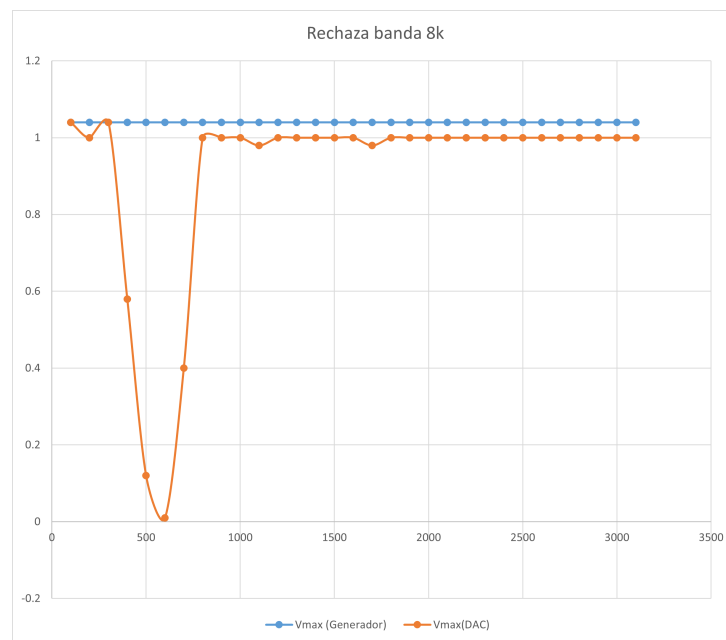


Figura 22: Respuesta en amplitud del filtro rechaza banda para una frecuencia de muestreo de 8 kHz.

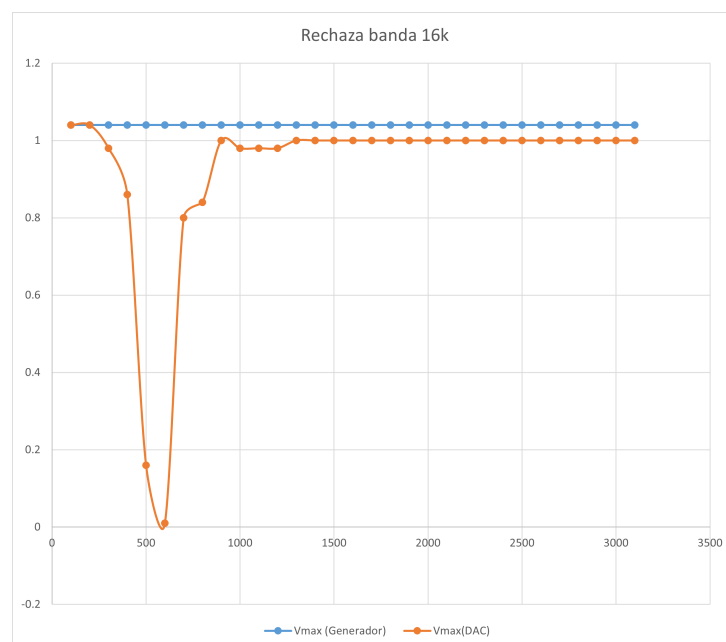


Figura 23: Respuesta en amplitud del filtro rechaza banda para una frecuencia de muestreo de 16 kHz.

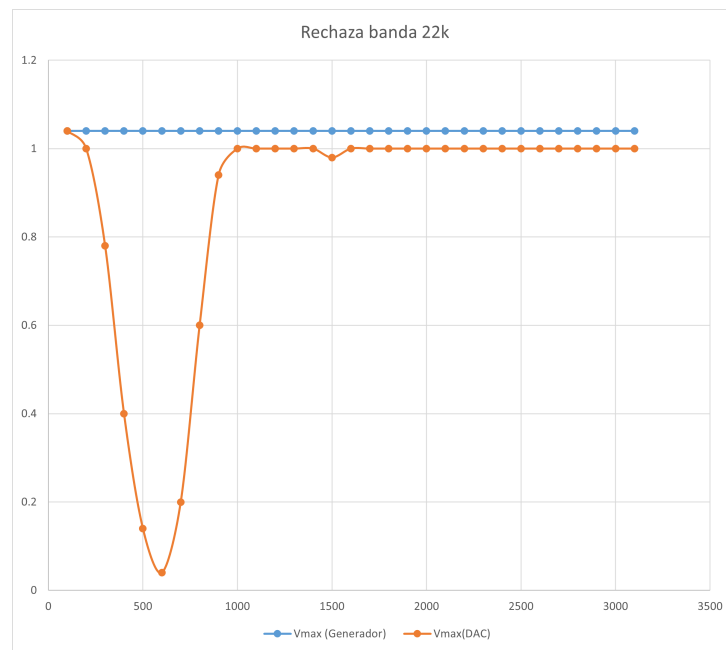


Figura 24: Respuesta en amplitud del filtro rechaza banda para una frecuencia de muestreo de 22 kHz.

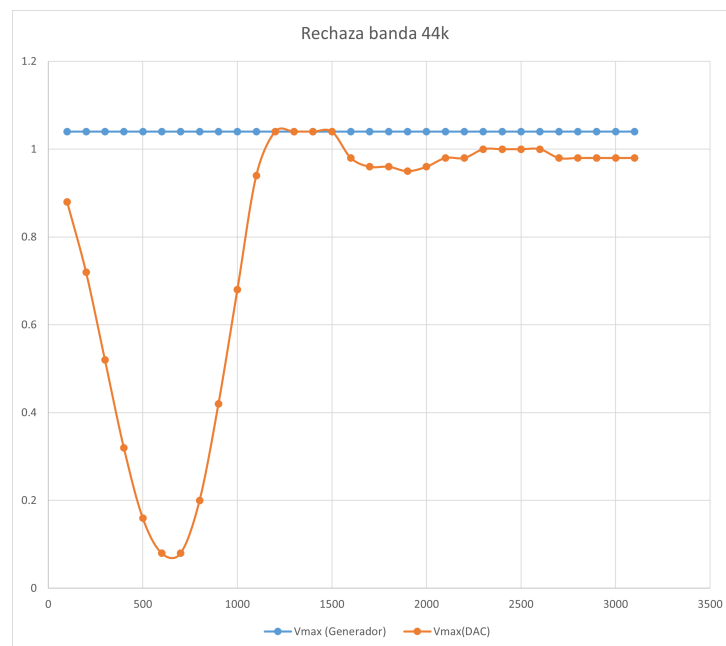


Figura 25: Respuesta en amplitud del filtro rechaza banda para una frecuencia de muestreo de 44 kHz.

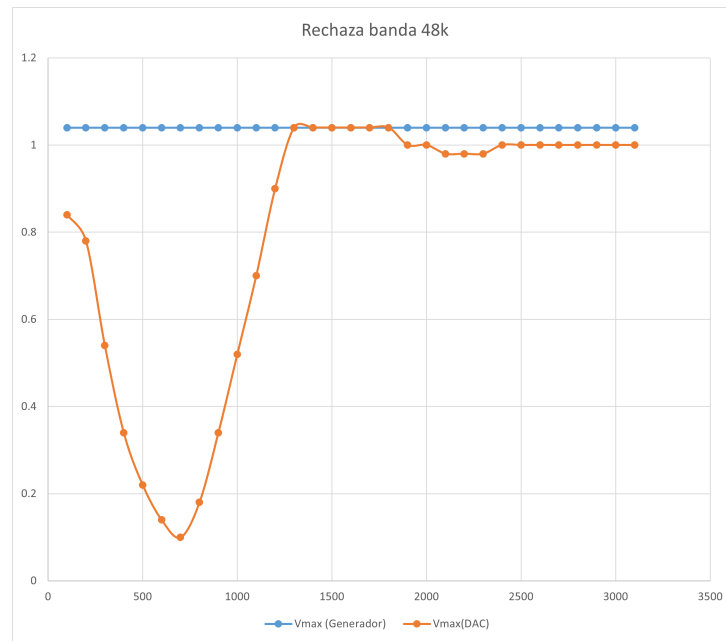


Figura 26: Respuesta en amplitud del filtro rechaza banda para una frecuencia de muestreo de 48 kHz.

8. Conclusiones

En el presente trabajo se realizó el diseño e implementación de filtros digitales FIR utilizando procesamiento por muestras sobre la plataforma STM32 Núcleo. Se desarrolló un programa capaz de procesar en tiempo real un buffer de 512 muestras provenientes del sistema de adquisición implementado en el Laboratorio N°1, permitiendo aplicar distintos tipos de filtros.

A partir de los resultados experimentales obtenidos mediante mediciones con osciloscopio, se verificó el correcto funcionamiento de los filtros diseñados. Las respuestas en amplitud observadas presentan el comportamiento esperado para cada tipo de filtro implementado: el filtro pasa bajos atenúa las componentes de frecuencia superiores a la frecuencia de corte especificada, el filtro pasa altos atenúa las componentes de baja frecuencia, el filtro pasa banda permite el paso de frecuencias dentro del rango definido, y el filtro rechaza banda reduce significativamente las componentes correspondientes a la banda de frecuencia especificada.

Asimismo, se comprobó que la modificación de la frecuencia de muestreo produce el corrimiento correspondiente de la respuesta en frecuencia en términos absolutos, manteniendo la forma característica de cada filtro cuando se analiza en frecuencia normalizada. Este resultado es consistente con los fundamentos teóricos del procesamiento digital de señales y valida el procedimiento de diseño de coeficientes realizado previamente.

El procesamiento por muestras demostró ser adecuado para la implementación en tiempo real, permitiendo la ejecución continua del algoritmo sin pérdidas de datos ni interrupciones en la señal de salida. El uso de buffers independientes para entrada y salida permitió implementar correctamente el modo bypass, facilitando la comparación directa entre la señal original y la señal filtrada.

Las diferencias observadas entre los valores teóricos y los resultados experimentales pueden atribuirse principalmente a efectos de cuantización de los coeficientes en formato Q15, resolución finita de los conversores ADC y DAC, y limitaciones propias de los instrumentos de medición. No obstante, los resultados obtenidos presentan una concordancia satisfactoria con lo esperado teóricamente.

En conclusión, el trabajo realizado permitió verificar experimentalmente la correcta implementación de filtros FIR en un sistema embebido, demostrando la viabilidad de aplicar técnicas de procesamiento digital de señales en tiempo real utilizando recursos de hardware limitados. Asimismo, se logró integrar los conceptos teóricos de diseño de filtros con su implementación práctica, validando el funcionamiento del sistema mediante mediciones experimentales.